

Fine-tuning Introduction

Jin Fulong

University of Science and Technology of China

April 2, 2025

Table of Contents

Introduction

SFT (Supervised Fine-tuning)

Different Structure

COSINE Framework (Weakly Supervised Fine-tuning))

LoRA

Intrinsic Dimensionality

LoRA on Transformer

RAG (Retrieval-Augmented Generation)

Hyperparameter

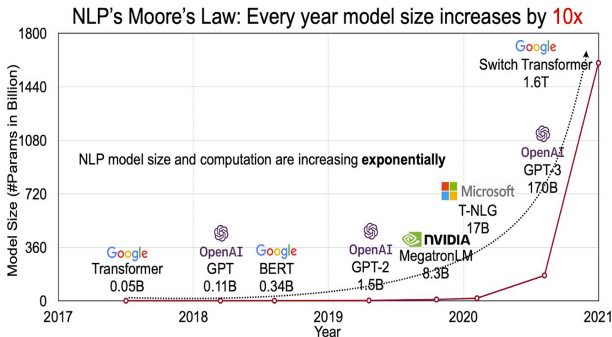
In modern Deep learning tasks like NLP, CV, we just need to feed black box function $f(x, \theta)$ with some **training dataset** $\{x_i, y_i\}_{i=1}^N$, making them work on some other prediction.

- Optimizer choice: SGD, ADAM (others like RMSprop, Adagrad, Nadam) specify the way to solve

$$\min_{\theta} \frac{1}{N} \sum_{i=1} R(f(x_i, \theta), y_i)$$

- Learning Rate adjustment: Too high or too low learning rate will lead a bad result on training model
- Regulation strategy (Weight Decay, Dropout): Helping prevent overfitting by decrease $|\theta|$ number or change $L(\theta) = R(\theta) + \lambda ||\theta||$

Facing problem



What about we have additional **input** for some downstream task for LLM like

- ▶ Chatting
- ▶ Translation
- ▶ Text to image, video.... ?

We can not just train a model specified for these specified tasks!

Solution (Prompt)

- In-context learning
 1. feed training examples (More specifically) to the LLM as a prompt.
 2. allow the LLM to infer patterns in the training examples during inference (i.e. decoding)
- **Pro:** Don't need backpropagation to update θ
- **Con:** Still costly as a result of $O(n^2)$ time complexity (n is max sequence length)

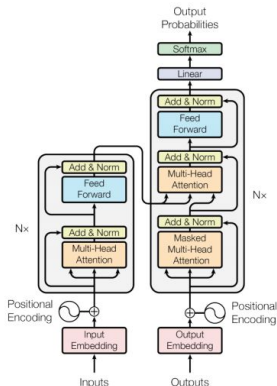


Figure: GPT

Example of Prompt

当有人让你给评价一下统计学四大期刊中的Biometrika的话，你会怎么火力全开的评论。让我给你个印信的模板例如，先使用最优雅的语言总结它如“统计学的兵马俑博物馆”，然后解释它。再使用新颖的方法总结其他优点，明白了吗？

现在评价一下JASA

请帮我给评价一下统计学四大期刊中的Biometrika，不要太长

🔍 **Biometrika**——统计学的“老钱”期刊。随着理论深入，数学越来越重要，但越来越像象牙塔里的自嗨。

优点:

- **理论逼格拉满**：证明必须优雅，数学必须硬核，适合传统派统计学家。
- **百年招牌**：发一篇前驱论，尤其在统计学术圈，老派教授会这个。

缺点:

- **应用？不存在**：数据驱动的研究家去JASA，这里只收“数学艺术品”。
- **审稿太慢**：6个月起步，审稿人可能为某个引理不够美而拒稿。
- **被卷麻了**：高维统计、ML方向干不过RSS-B和Annals，死守传统阵地。

总结：统计学的“古典乐”——高贵但枯燥和慢，适合理论圈数独主义者，应用派请绕道。

🔍 **Biometrika**脱评模板

1. “统计学的兵马俑博物馆”

——理论够古老，地位够崇高，但创新全靠考古。发一篇延迟学术陈腐，但想卖点现代统计？出门左转RSS-B，右拐JASA。

2. “数学家的八股文考场”

——证明不优雅？拒！定理不够美？拒！审稿人拿放大镜找瑕疵。6个月告诉你“引用3的瑕疵不够古典”，真当时髦的MCMC，直接统计不存在？

3. “选理路上的咬文嚼字”

——生物统计起家，现在连生物学家都懒得看。数据驱动？实证分析？醒醒，这儿只收“只道是”理论论文，应用建议投《Nature》子刊，别来沾边。

4. “数论学者的快乐老家”

——牛津老钱风，圈子味浓厚。论点再承关系或者数据学背景？拒你和“缺乏深刻性”的瑕疵和审稿年好合。

🔍 **JASA**脱评模板

1. “统计学的华尔街”

——影响因子就是硬通货，管你理论还是应用派，数据商引就靠它。发一篇顶刊论文，但想追求学术纯粹？出门左转《Annals》，右拐《Biometrika》。

2. “大数据的炼丹炉”

——从贝叶斯到机器学习，从因果推断到社交网络分析，只要包够fancy，啥都能往里塞。审稿人自己可能看不懂你的方法，但只要“看起来有用”，就通过。

3. “审稿人的迷宫”

——3个月给你寄过拒稿的小修，也可给6个月告诉你“缺乏创新”（其实是审稿人忙碌和老板吵架了），运气差点，纯看命。

4. “学术工业化的标杆”

——流水线式发文章，适合急需tenure的青年和只想堆砌方法的赌王。真正的理论突破？别闹了，这儿流行的是“新瓶装旧酒+豪华实验包装”。

Figure: before prompt

Figure: prompting

Figure: after prompt

Solution (fine-tuning)

Large Language Model

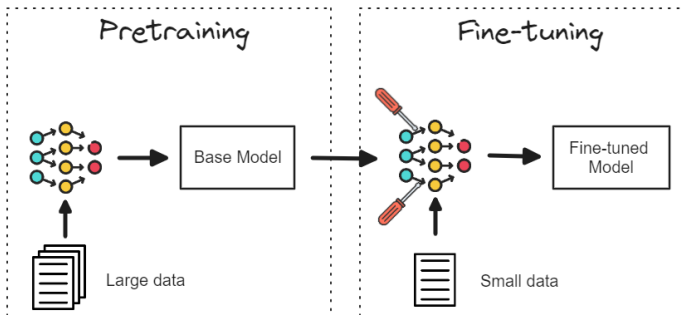


Figure: Fine-tuning process

Some examples of downstream tasks

► Named Entity Recognition (NER):

text (x): Gates creates Microsoft in Seattle,
labels (y): ["B-PER", "O", "B-LOC", "O", "B-ORG"]

► Stanford Question Answering Dataset (SQuAD)

text (x): BERT is a pretrained structure proposed by Google,
question (x): who propose BERT?,
answer (y): text: Google, start: 8, end: 8

Smaller samples provided is also called few-shot setting

Three different types

Fine-tuning contains three different types including

- ▶ **Supervised Fine-Tuning, (SFT):** with additional data $\{x_i, y_i\}_{i=1}^N$ to update $\theta_0 \subset \theta$, dealing SQuAD, NER so on
- ▶ **Unsupervised Fine-Tuning:** with additional data $\{x_i\}_{i=1}^N$ to update $\theta_0 \subset \theta$, dealing with MLM...
- ▶ **Weakly-Supervised Fine-Tuning:** combinations of two above situations, focusing on robustness performance and denoising

There also Pros and Cons for it

- ▶ **Pro:** Fitting into the standard ML recipe and efficient when N is big
- ▶ **Cons:** Might not have access to the model at all

Example of (SFT)

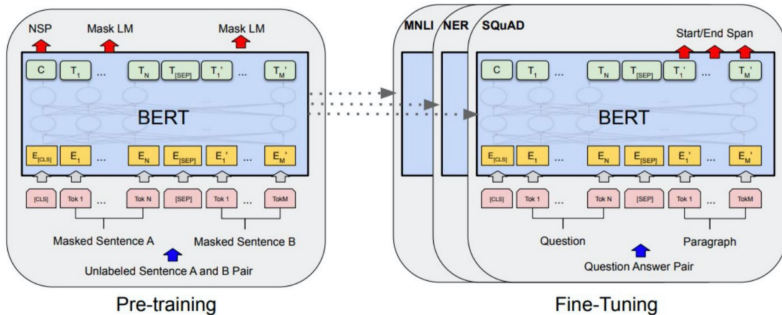


Figure: Different Fine-tuning task of Bert

Power of Fine-tuning

- ▶ MNLI (Multi-Genre Natural Language Inference): relationship between a pair of sentences (a **Premise** and a **Hypothesis**), it is classified into three categories——(**Entailment**, **Contradiction**, **Neutral**)
- ▶ RTE (Recognizing Textual Entailment): Just judge if there is **Entailment** between two adherent sentences.

Method	MNLI-m (Val. Acc./%)	RTE (Val. Acc./%)
GPT-3 Few-Shot	40.6	69.0
GPT-3 Fine-Tuned	89.5	85.4

Figure: Comparisons

Prompt v.s. Fine-tuning

In most situations, Fine-tuning works better than In-Context Learning (GPT-3)

		FT									FT						
		125M	350M	1.3B	2.7B	6.7B	13B	30B			125M	350M	1.3B	2.7B	6.7B	13B	30B
ICL	125M	-0.00	0.01	0.02	0.03	0.12	0.14	0.09	ICL	125M	-0.00	0.00	0.02	0.01	0.10	0.11	0.07
	350M	-0.00	0.01	0.02	0.03	0.12	0.14	0.09		350M	-0.00	0.00	0.02	0.01	0.10	0.11	0.07
	1.3B	-0.00	0.01	0.02	0.03	0.12	0.14	0.09		1.3B	-0.01	-0.00	0.01	0.01	0.10	0.11	0.07
	2.7B	-0.00	0.01	0.02	0.03	0.12	0.14	0.09		2.7B	-0.01	-0.00	0.01	0.01	0.09	0.10	0.07
	6.7B	-0.00	0.01	0.02	0.03	0.12	0.14	0.09		6.7B	-0.01	-0.01	0.01	0.00	0.09	0.10	0.06
	13B	-0.04	-0.02	-0.01	-0.00	0.09	0.11	0.05		13B	-0.03	-0.03	-0.02	-0.02	0.07	0.08	0.04
	30B	-0.11	-0.09	-0.08	-0.08	0.02	0.03	-0.02		30B	-0.07	-0.07	-0.05	-0.06	0.03	0.04	0.00
(a) RTE									(b) MNLI								

Table 1: Difference between average **out-of-domain performance** of ICL and FT on RTE (a) and MNLI (b) across model sizes. We use 16 examples and 10 random seeds for both approaches. For ICL, we use the `gpt-3` pattern. For FT, we use pattern-based fine-tuning (PBFT) and select checkpoints according to in-domain performance. We perform a Welch's t-test and color cells according to whether: **ICL performs significantly better than FT**, **FT performs significantly better than ICL**. For cells without color, there is no significant difference.

Table of Contents

Introduction

SFT (Supervised Fine-tuning)

Different Structure

COSINE Framework (Weakly Supervised Fine-tuning))

LoRA

Intrinsic Dimensionality

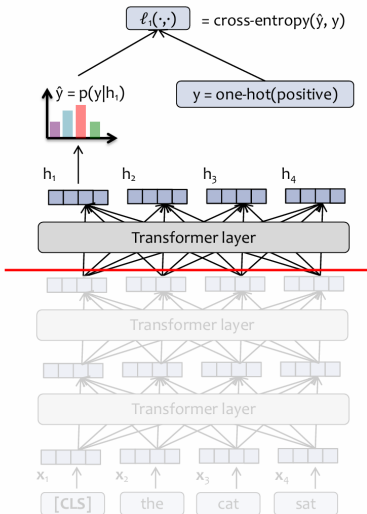
LoRA on Transformer

RAG (Retrieval-Augmented Generation)

Top K layer Only

- ▶ Keep all parameters fixed except for the top K layers—gradients only need to flow through K layers instead of full layers.
- ▶ Reduced memory usage we don't need to store the adjoints (gradient of the loss with respect to each parameter) for the full computation graph.

stop gradient here
s.t. error does not
backprop to lower
layers



Prefix-embedding tuning

This technology injecting dummy tokens in **front, middle and back** position of a sentence, for example

► Original:

'I', 'Like', 'the', 'cat'

► front:

'pi', 'pi', 'I', 'Like', 'the', 'cat'

prefix

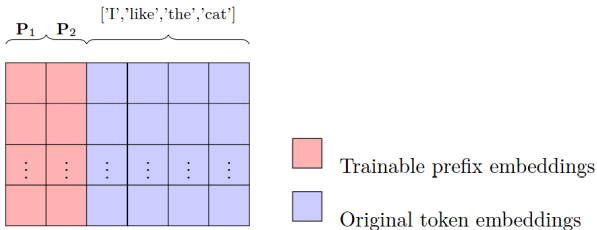


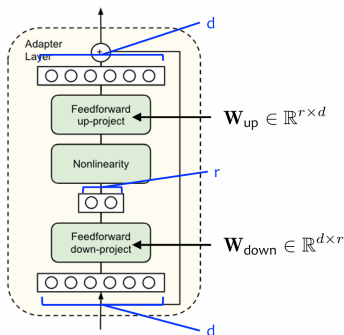
Figure: transformer embedding of K, V

Adaptive Layer

- ▶ An adapter layer is simply a feed forward neural network with one hidden layer, and a residual connection with $r \ll d$

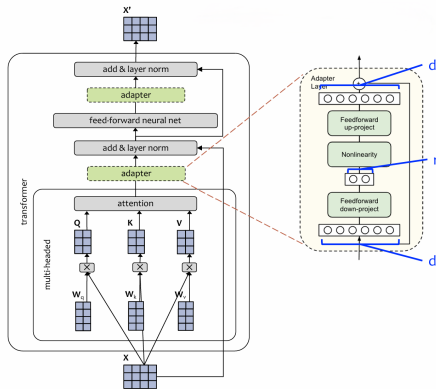
$$\text{Adapter}(x) = x + W_{\text{up}} \cdot \text{ReLU}(W_{\text{down}} \cdot x)$$

- ▶ when $d = 64$, Adapter only optimize 0.1% of whole $|\theta|$, saving more than 90% memory



Adaptive layers for Transformer

- ▶ When added to a deep neural network (e.g. Transformer) all the other parameters of the pretrained model are kept fixed, and only the adapter layer parameters are fine-tuned
- ▶ Interesting: it works even though the grey modules are kept fixed!



Comparison

It seems existing a trade off between two methods

- ▶ **Top K** : With more parameters, achieving better accuracy
- ▶ **Adapters**: Despite with less trainable parameters, it can't outperform than **Top K**

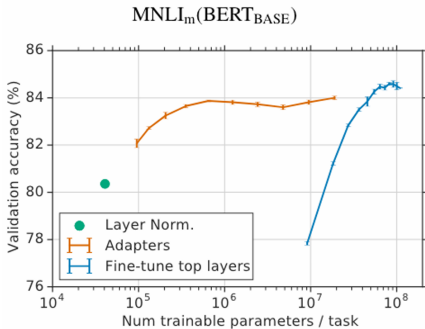


Figure: Top K Layer vs Adapters on MNLI

Intuition

During fine-tuning, the semantic and syntactic knowledge in the pre-trained LMs is adapted for the target task works well.

- **Labeled data insufficient**: one bottleneck for fine-tuning LMs is the requirement of labeled data

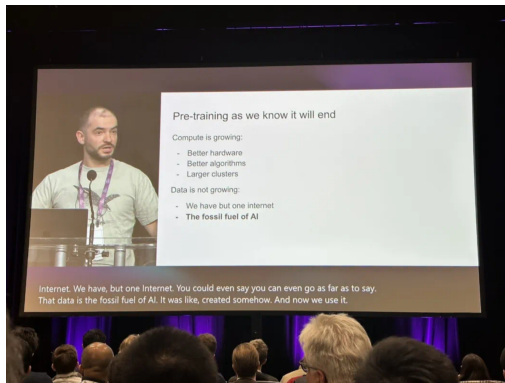


Figure: Ilya's speech on NIPS 2024

Self training

Self-training (Rosenberg et al., 2005; Lee, 2013) is a proper tool for fine-tuning language models with weak supervision. It augments the training set with unlabeled data by generating pseudo-labels for them, which improves the models' generalization power,

- **Pesudo-label:** assume we have a pretrained model $f(x, \theta)$ with $\mathcal{D} = \{x_i, y_i\}_{i=1}^N$, for unlabeled $\{z_{i=1}^K\}$, generating

$$\tilde{y}_i = f(z_i, \hat{\theta}(x, y))$$

- **Self-training:** Combine $\{x_i, y_i\}_{i=1}^N$ and $\{z_i, \tilde{y}_i\}_{i=1}^K$ retraining f

Cons: No information z_i bring noise for training, how to deal with it?

COSINE

- COSINE provides a criteria for generated samples $\{z_i, \tilde{y}_i\}_{i=1}^K$

$$\{x_i, \hat{y}_i\}_{i=1}^N \cup \{z_i, \tilde{y}_i\}_{i=1}^K = \text{High-conf} \cup \text{Low-conf}$$

- For classified tasks, High-conf utilize penalization R_2 to keep features more 'concentrated'.

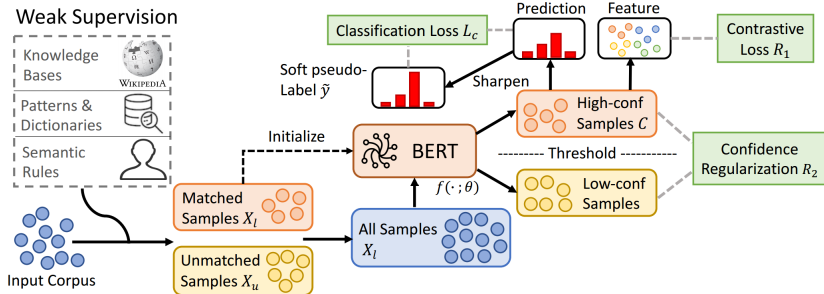


Figure: COSINE'S Framework

COSINE

Now, we deal with Sequence Classification, Token Classification, Sentence Pair Classification, Hence for $y \in \mathcal{Y}$ belongs to a class $\mathcal{Y} = \{1, 2, \dots, C\}$

► **Pretrained Model Loss:**

$$\min_{\theta} \frac{1}{|\mathcal{X}_l|} \sum_{(x_i, y_i) \in \mathcal{X}_l} \text{CE}(f(x_i; \theta), y_i),$$

► **Soft pseudo-labels:** in a batch $\mathcal{B}$, use a non binary response in $[0,1]$

$$\tilde{y}_j = \frac{[f(x; \theta)]_j^2 / f_j}{\sum_{j' \in \mathcal{Y}} [f(x; \theta)]_{j'}^2 / f_{j'}} \in \mathbb{R}^C,$$

where $f_j = \sum_{x' \in \mathcal{B}} [f(x'; \theta)]_j^2$

COSINE (continued)

- **Sample Reweighting:** For a pseudo-labels

$$\omega = 1 - \frac{H(\tilde{\mathbf{y}})}{\log(C)}, H(\tilde{\mathbf{y}}) = - \sum_{i=1}^C \tilde{\mathbf{y}}_i \log \tilde{\mathbf{y}}_i,$$

Bigger w means more information on classification, hence more potential in
High-conf $\mathcal{C} = \{x \in \mathcal{B} \mid \omega(x) \geq \xi\}$ given threshold ξ .

- **Fine-tuning Loss:**

$$\mathcal{L}_c(\theta, \tilde{\mathbf{y}}) = \frac{1}{|\mathcal{C}|} \sum_{x \in \mathcal{C}} \omega(x) \mathcal{D}_{\text{KL}}(\tilde{\mathbf{y}} \| f(x; \theta)),$$

- **Confidence regularization:** For controlling $|\mathcal{C}|$ not too much, use

$$\mathcal{R}_2(\theta) = \frac{1}{|\mathcal{C}|} \sum_{x \in \mathcal{C}} \mathcal{D}_{\text{KL}}(\mathbf{u} \| f(x; \theta))$$

where $\mathbf{u}_i = 1/C, i = 1, \dots, C$

COSINE (continued)

- **Contrastive Learning on Sample Pairs:** take $\mathbf{v} = \text{BERT}(x) \in \mathbb{R}^d$

$$\mathcal{R}_1(\theta; \tilde{\mathbf{y}}) = \sum_{(x_i, x_j) \in \mathcal{C} \times \mathcal{C}} \ell(\mathbf{v}_i, \mathbf{v}_j, W_{ij}),$$

where $W_{ij} = I(\arg \max \tilde{\mathbf{y}}_i = \arg \max \tilde{\mathbf{y}}_j)$,

$$\ell = W_{ij} d_{ij}^2 + (1 - W_{ij}) [\max(0, \gamma - d_{ij})]^2.$$

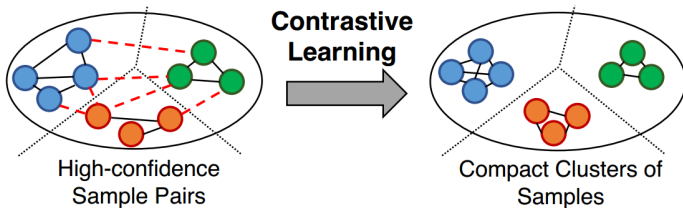


Figure: The effect on Contrastive Learning

Robustness Against Label Noise

- We corrupt certain percentage of labels by randomly changing each one of them to another class on **TRECT** dataset

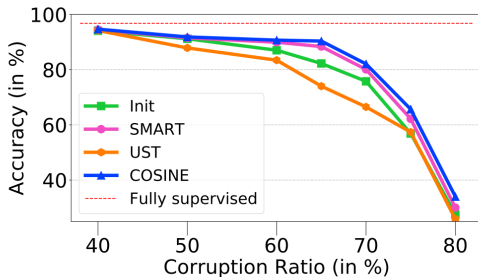


Figure 3: *Results of label corruption on TREC. When the corruption ratio is less than 40%, the performance is close to the fully supervised method.*

- Compared with advanced **fine-tuning** and **self-training** methods (e.g. SMART and UST) This model consistently outperforms the baselines.

Semi-supervise Learning

- ▶ **labeled dataset source** : Wic dataset
- ▶ **unlabeled dataset source**: Lexical databases (e.g., WordNet, Wictionary)

Semi-Supervised Learning			
SenseBERT (Levine et al., 2020)	—	72.1	370M
RoBERTa-WL [†] (Liu et al., 2019)	72.3	70.2	125M
w/ MT [†] (Tarvainen and Valpola, 2017)	73.5	70.9	125M
w/ VAT [†] (Miyato et al., 2018)	74.2	71.2	125M
w/ COSINE [†]	76.0	73.2	125M

Figure: Performance Benchmark

Table of Contents

Introduction

SFT (Supervised Fine-tuning)

Different Structure

COSINE Framework (Weakly Supervised Fine-tuning))

LoRA

Intrinsic Dimensionality

LoRA on Transformer

RAG (Retrieval-Augmented Generation)

Intrinsic Dimensionality

The intrinsic dimension of an objective function measures the minimum number of parameters $|\theta|$ needed to reach satisfactory solutions to the respective objective (accuracy).

- ▶ Let the intrinsic dimension be the value of d when good solutions (above 90% threshold of full parameterization) start to appear, it's so-called d_{90}

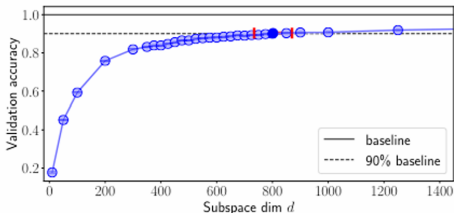


Figure: For MNIST digit recognition, original neural network has $|\theta|=199,210$ parameters but $d_{90}=750$

Intrinsic Dimensionality

- RoBERTa is augmented model with a larger dataset, including BookCorpus, English Wikipedia, CC-NEWS and so on.

1. Base : 110 billion parameters
2. LARGE : 345 billion parameters

$$\theta^{(D)} = \theta_0^{(D)} + P_{\text{SAID or DID}} \theta^{(d)}, \quad \text{where } P \in \mathbb{R}^{D \times d}$$

Model	SAID		DID	
	MRPC	QQP	MRPC	QQP
BERT-Base	1608	8030	1861	9295
BERT-Large	1037	1200	2493	1389
RoBERTa-Base	896	896	1000	1389
RoBERTa-Large	207	774	322	774

Figure: d_{90} under two different compressed calculations

LoRA

- Keep the original pretrained parameters $\mathbf{W}_0$ fixed during fine-tuning
- Learn an additive modification to those parameters $\Delta\mathbf{W}$
- Define $\Delta\mathbf{W}$ via a low rank decomposition:

$$\Delta\mathbf{W} = \mathbf{BA}$$

where $\mathbf{BA}$ has rank r , which is much less than the input dimension k or the output dimension

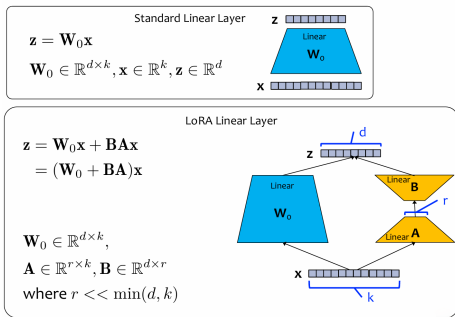


Figure: what's LoRA

LoRA

- ▶ To get a Standard Linear Layer with parameters $\mathbf{W}$ that includes our LoRA fine tuning:

$$\mathbf{W} \leftarrow \mathbf{W}_0 + \mathbf{B}\mathbf{A}$$

- ▶ To remove the LoRA fine tuning from that Standard Linear Layer:

$$\mathbf{W} \leftarrow \mathbf{W} - \mathbf{B}\mathbf{A} = \mathbf{W}_0$$

- ▶ If we do LoRA training on two tasks s.t. the parameters $\mathbf{B}'\mathbf{A}'$ are for task 1 and $\mathbf{B}''\mathbf{A}''$ are for task 2, then we can swap back and forth between them

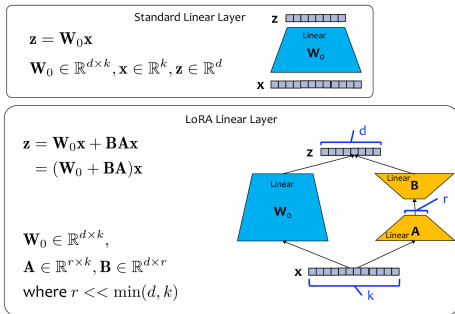


Figure: what's LoRA

LoRA

- We initialize the trainable parameters:

$$A_{ij} \sim \mathcal{N}(0, \sigma^2), \forall i, j$$

$$\mathbf{B} = 0$$

- This ensures that, at the start of fine tuning, the parameters have their pretrained values

$$\Delta \mathbf{W} = \mathbf{B}\mathbf{A} = 0$$

$$\mathbf{W}_0 + \mathbf{B}\mathbf{A} = \mathbf{W}_0$$

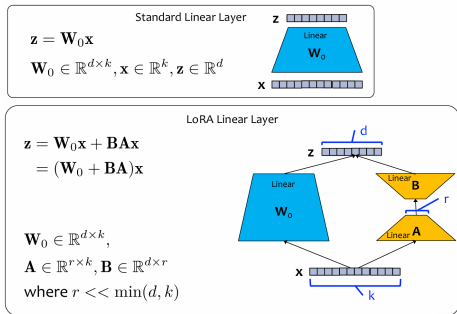
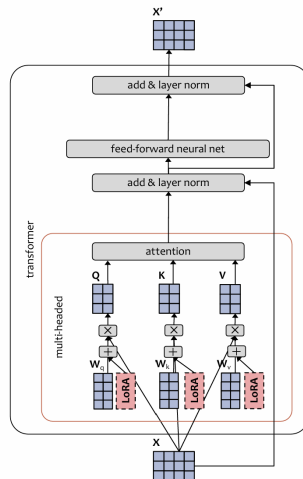


Figure: what's LoRA

LoRA for Transformer

- Empirically, for GPT-3, they also find that it is most efficient to include LoRA only on the query K and value V linear layers

	# of Trainable Parameters = 18M							
Weight Type	W_q	W_k	W_v	W_o	W_q, W_k	W_q, W_v	W_q, W_k, W_v, W_o	
Rank r	8	8	8	8	4	4	2	
WikiSQL ($\pm 0.5\%$)	70.4	70.0	73.0	73.2	71.4	73.7	73.7	
MultiNLI ($\pm 0.1\%$)	91.0	90.8	91.0	91.3	91.3	91.3	91.7	



Benchmark comparison

- ▶ **Fine-Tuning:** just the last two layers (FT-Top2).
- ▶ **Bias only:** only train the bias vectors when adapting to downstream tasks

$$\phi \cdots \phi \odot \phi(\mathbf{W}_0 x + \underbrace{b}_{\text{trainable}}),$$

- ▶ **Prefix-embedding tuning:** injects special tokens alongside the input tokens which trainable parameter number

$$|\theta| = d_{\text{model}} \times (l_p + l_i),$$

- ▶ **Adapter tuning:** inserts adapter layers between the self-attention module which trainable parameter number

$$|\theta| = 4 \times L \times d_{\text{model}} \times r.$$

Performance on GPT3

- with a batch size of 100k tokens, a sequence length of 768, and a weight decay factor of 0.1 tune the learning rate for all method-dataset combinations

Method	# of Trainable Parameters	WikiSQL	MNLI-m	SAMSum
		Accuracy (%)	Accuracy (%)	R1/R2/RL
GPT-3 175B (Fine-Tune)	175,255.8M	73.0	89.5	52.0/28.0/44.5
GPT-3 175B (Bias Only)	14.2M	71.3	91.0	51.3/27.4/43.5
GPT-3 175B (PrefixEmbed)	3.2M	63.1	88.6	48.3/24.2/40.5
GPT-3 175B (PrefixLayer)	20.2M	70.1	89.5	50.8/27.3/43.5
GPT-3 175B (LoRA)	4.7M	73.4	91.3	52.1/28.3/44.3
GPT-3 175B (LoRA)	37.7M	73.8	91.7	53.2/29.2/45.0

Table 1: Logical form validation accuracy on WikiSQL, validation accuracy on MultiNLI-matched and Rouge-1/2/L on SAMSum achieved by different GPT-3 adaptation methods. LoRA performs better than prior approaches, including conventional fine-tuning. The result on WikiSQL has a fluctuation of $\pm 0.3\%$ and MNLI-m $\pm 0.1\%$.

Choice of Optimal Rank r

- Although a accuracy testing on different r showing that $r \ll d_{model}$, how can we make sure low-rank adaptation matrix is sufficient?

	Weight Type	$r = 1$	$r = 2$	$r = 4$	$r = 8$	$r = 64$
WikiSQL($\pm 0.3\%$)	W_q, W_v	73.4	73.3	73.7	73.8	73.5
	W_q	68.8	69.6	70.5	70.4	70.0
MultiNLI ($\pm 0.1\%$)	W_q, W_v	91.3	91.4	91.3	91.7	91.4

Table 4: Validation accuracy on WikiSQL and MultiNLI with different rank r . To our surprise, a rank as small as one suffices for adapting both W_q and W_v on these datasets while training W_q alone needs a larger r . We replicate this on GPT-2 in Sec. G.2.

Subspace similarity between different r

We answer the question by this way

- ▶ assume U_{r_1} and U_{r_2} are right singular vector from SVD PF A_{r_1} and A_{r_2}

$$A_r = U_r \Sigma_r V_r^T$$

- ▶ if top r_1 singular value's vector sub-space is contained in top r_2 singular value's s vector sub-space, that's make sense

$$\text{span}(U_{r_1} \text{'s top } r_1 \text{ columns}) \subset \text{span}(U_{r_2} \text{'s top } r_2 \text{ columns})$$

- ▶ Grassman distance can deal with this correlation

$$\phi(A_{r_1}, A_{r_2}, i, j) = \frac{\|U_{A_{r_1}}^{i\top} U_{A_{r_2}}^j\|_F^2}{\min(i, j)} \in [0, 1]$$

Subspace similarity between different r

- Directions corresponding to the top singular vector overlap significantly between $A_r = 8$ and $A_r = 64$, while others do not. Specifically, ΔW_v (resp. ΔW_q) of $A_r = 8$ and ΔW_v (resp. ΔW_q) of $A_r = 64$ share a subspace of dimension 1 with normalized similarity > 0.5

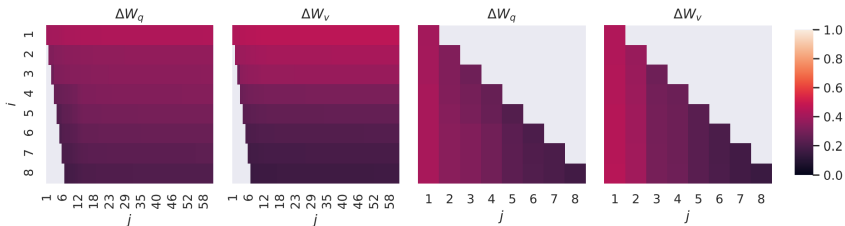


Figure 3: Subspace similarity between column vectors of $A_{r=8}$ and $A_{r=64}$ for both ΔW_q and ΔW_v . The third and the fourth figures zoom in on the lower-left triangle in the first two figures. The top directions in $r = 8$ are included in $r = 64$, and vice versa.

Table of Contents

Introduction

SFT (Supervised Fine-tuning)

Different Structure

COSINE Framework (Weakly Supervised Fine-tuning))

LoRA

Intrinsic Dimensionality

LoRA on Transformer

RAG (Retrieval-Augmented Generation)

Intuition

- ▶ No matter **Few shoting, fine-tuning, LoRA**, we are all doing one actual thing

$$\theta \leftarrow \theta_0 + \alpha \nabla R, \quad \text{where } \theta \text{ only part of whole parameter}$$

- ▶ **Cons:**

1. The knowledge transference to numerics can't straightforwardly provide insight into their predictions, and may produce "hallucinations"
2. over-dependent on extra Knowledge input for special or academic tasks.

Pipeline

RAG combined pretrained **Non-Parametric** and **Parametric** to achieve “hallucinations”

- **Non-Parametric Module:** A Query system such Google, Bing and it gives us top K relative document $\{z_1, \dots, z_K\}$
- **Parametric Module:** Feed such relative $\{z_1, \dots, z_K\}$ to model f

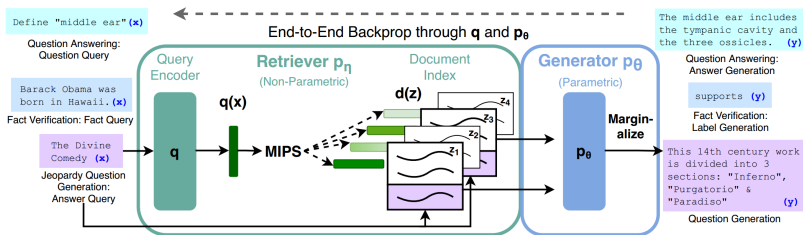


Figure: Pipeline of RAG

Method Details

► RAG-Sequence Model: One times searching

$$\begin{aligned} p_{\text{RAG-Sequence}}(y \mid x) &\approx \sum_{z \in \text{top } k(p(\cdot \mid x))} p_{\eta}(z \mid x) p_{\theta}(y \mid x, z) \\ &= \sum_{z \in \text{top } k(p(\cdot \mid x))} p_{\eta}(z \mid x) \prod_i^N p_{\theta}(y_i \mid x, z, y_{1:i-1}) \end{aligned}$$

► RAG-Token Model: Multi times searching

$$p_{\text{RAG-Token}}(y \mid x) \approx \prod_i^N \sum_{z \in \text{top } -k(p(\cdot \mid x))} p_{\eta}(z \mid x) p_{\theta}(y_i \mid x, z, y_{1:i-1})$$

Jeopardy Question Generation

Jeopardy Question Generation is an unusual format that consists of trying to guess an entity from a fact about that entity.

- ▶ **Input:** an answer of possible question
- ▶ **Output:** a reasonable question according to the answer

For example when given a answer

World Cup.

A good question is

what international sports competition Mexico scored as the first country to host twice?

Performance of RAG

- ▶ '?' indicates factually incorrect responses,
- ▶ * indicates partially correct responses

Jeopardy Question Gener- ation	Washington	BART	?This state has the largest number of counties in the U.S.
		RAG-T	It's the only U.S. state named for a U.S. president
		RAG-S	It's the state where you'll find Mount Rainier National Park
	The Divine Comedy	BART	*This epic poem by Dante is divided into 3 parts: the Inferno, the Purgatorio & the Purgatorio
		RAG-T	Dante's "Inferno" is the first part of this epic poem
		RAG-S	This 14th century work is divided into 3 sections: "Inferno", "Purgatorio" & "Paradiso"

Figure: Answer differences between RAG and Pretrained model

Reference



Yu, Y., Zuo, S., Jiang, H., Ren, W., Zhao, T., & Zhang, C. (2020). Fine-tuning pre-trained language model with weak supervision: A contrastive-regularized self-training approach. arXiv preprint arXiv:2010.07835.



Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., ... & Kiela, D. (2020). Retrieval-augmented generation for knowledge-intensive nlp tasks. Advances in neural information processing systems, 33, 9459-9474.



Hu, E. J., Shen, Y., Wallis, P., Allen-Zhu, Z., Li, Y., Wang, S., ... & Chen, W. (2022). Lora: Low-rank adaptation of large language models. ICLR, 1(2), 3.



Li, X. L., & Liang, P. (2021). Prefix-tuning: Optimizing continuous prompts for generation. arXiv preprint arXiv:2101.00190.



Houlsby, N., Giurgiu, A., Jastrzebski, S., Morrone, B., De Laroussilhe, Q., Gesmundo, A., ... & Gelly, S. (2019, May). Parameter-efficient transfer learning for NLP. In International conference on machine learning (pp. 2790-2799). PMLR.